Week 5: Value Function Approximation

Seasons of Code — Reinforcement Learning

Function Approximation and Deep Q-Networks

Function Approximation in RL

Instead of a tabular representation, we approximate the action-value function using a parameterized function

$$Q(s, a; \theta) \approx Q^*(s, a),$$

where θ denotes the parameters of the approximator (weights of a linear model or a neural network). This is necessary when the state space is large or continuous, since a lookup table $Q(s, a)$ becomes infeasible to store or learn.

Linear vs Nonlinear Approximators

For a **linear approximator**, we write

$$Q(s, a; \theta) = \phi(s, a)^\top \theta,$$

where $\phi(s, a)$ is a hand-crafted feature vector. Linear approximators have convex loss landscapes and are relatively stable under gradient descent, but are limited by the expressiveness of ϕ.

For a **nonlinear approximator** (e.g., a deep neural network as in DQN), $Q(s, a; \theta)$ is the output of a network with parameters θ. This gives far greater representational capacity but loses convergence guarantees and introduces instability when combined with bootstrapping and off-policy updates — the so-called *deadly triad* (function approximation + bootstrapping + off-policy learning).

DQN Loss

DQN minimizes the mean-squared temporal-difference (TD) error over transitions (s, a, r, s') sampled from a replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y - Q(s, a; \theta)\right)^2\right],$$

where the TD target y is computed using a separate **target network** with parameters θ^-:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

The gradient used for the update is

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y - Q(s, a; \theta)\right)\nabla_\theta Q(s, a; \theta)\right].$$

Note that the gradient is *not* taken through y, since θ^- is treated as fixed (a semi-gradient method).

Why Experience Replay Helps

Online RL generates a sequence of highly **correlated** transitions, since consecutive states s_t, s_{t+1} are temporally dependent. Training directly on this stream violates the i.i.d. assumption underlying stochastic gradient descent and causes:

- high variance, correlated gradient updates
- catastrophic forgetting of earlier experience
- oscillatory or divergent behavior

The **replay buffer** $\mathcal{D}$ stores past transitions (s, a, r, s') and samples minibatches uniformly at random during training. This:

- breaks temporal correlation between consecutive samples
- allows each transition to be reused multiple times, improving sample efficiency
- smooths the data distribution over many past policies rather than just the current one

Why the Target Network Helps

If a single network were used to compute both $Q(s, a; \theta)$ and the bootstrap target $\max_{a'} Q(s', a'; \theta)$, then every gradient step changes the target itself, since $\theta^- = \theta$. This creates a *moving target* problem: the optimization is chasing a target that shifts with every update, which can lead to oscillations or divergence, especially since $\max_{a'}$ introduces additional positive bias.

The **target network** freezes θ^- for C steps (or updates it slowly via Polyak averaging $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$), so that

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

remains stable over many gradient steps. This decouples the target computation from the parameters being optimized, effectively converting the problem into a sequence of stationary supervised regression problems and greatly improving training stability.